《人机交互技术实验》

实 验 报 告

学　　号 2024112835

姓　　名 王一龙

年　　级 2024 级

专　　业 软件工程

院　　系 计算机与人工智能学院

成绩	批阅人签字

2026-2027 学年第 1 学期

Git 操作实验

1. 实验目的

（1）学习 Git 的基本原理。

（2）掌握在命令行界面（CLI）下利用 Git 进行版本管理的方法。

2. 实验环境

2.1 硬件环境

（1） 计算机：ALIENWARE R13

（2） CPU:Intel(R) Core(TM) Ultra 9 185H (2.30 GHz)

（3） RAM：32GB

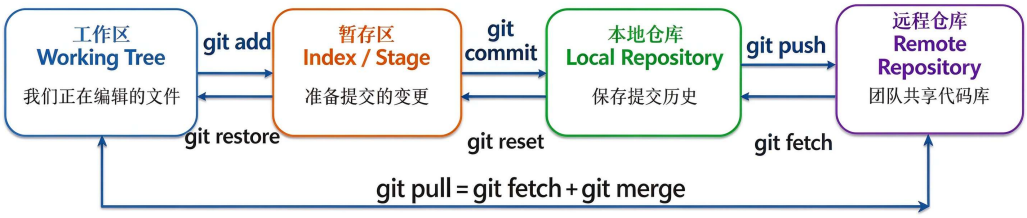
2.2 软件环境

（1） 操作系统：Windows11 家庭中文版

（2） 实验工具：Git-2.51.0-64-bit

3. 实验任务及结果

3.1 画出 Git 的结构图



（1）工作区（Working Directory）：用户当前编辑文件所在的目录，即本地磁盘上可以看到和修改的文件区域。

（2）暂存区（Staging Area / Index）：位于.git 目录中，用于临时存放执行 git add 后准备提交的改动，是工作区与本地仓库之间的中间层。

（3）本地仓库（Local Repository）：保存在.git 目录中的版本数据库，记录每一次 git commit 提交形成的完整历史快照。

（4）远程仓库（Remote Repository）：托管在服务器上的共享版本库（如 GitHub、Gitee），用于团队协作、数据备份与异地开发。

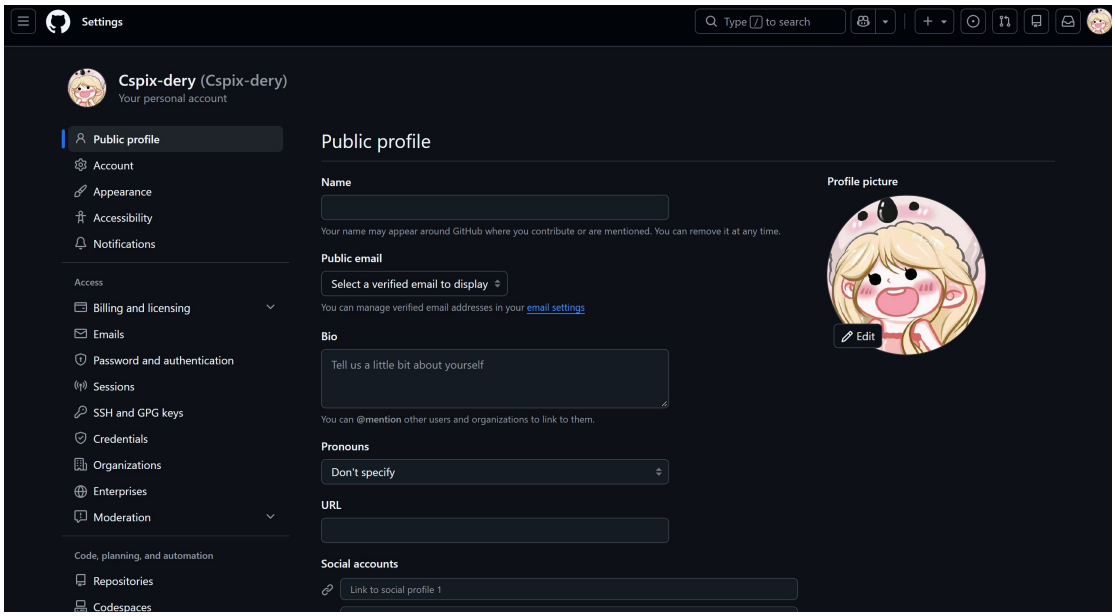
数据流向：工作区经 git add 进入暂存区，经 git commit 进入本地仓库，经 git push 进入远程仓库；反向通过 git clone 或 git pull 将远程内容取回本地，经 git checkout 恢复到工作区。

3.2 列出 Git 的常用命令

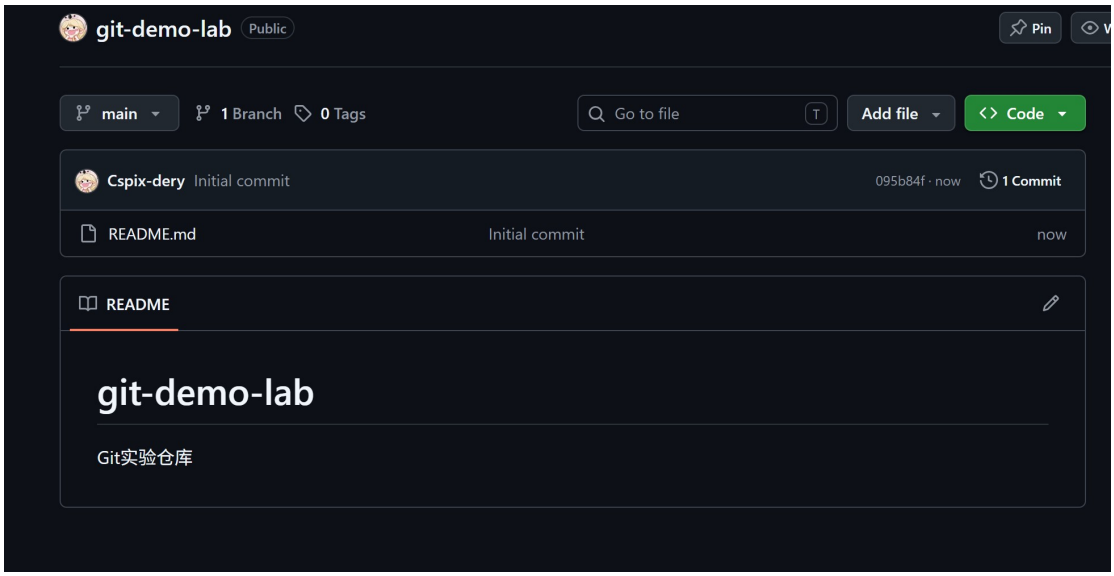
Git 命令	功能	示例
git init	初始化 git 库	git init
git status	查看工作区与暂存区的状态	git status
git add	将文件从工作区添加到暂存区	git add hello.txt
git commit -m	将暂存区内容提交到本地仓库	git commit -m “first commit”
git log	查看提交历史记录	git log --oneline
git branch	创建、查看或删除分支	git branch dev
git checkout	切换分支或恢复文件	git checkout dev
git merge	将指定分支合并到当前分支	git merge dev
git clone	克隆远程仓库到本地	git clone https://gitee.com/xxx/repo.git
git push	将本地提交推送到远程仓库	git push -u origin master
git remote add	关联远程仓库地址	git remote add origin https://gitee.com/xxx/repo.git
git pull	拉取远程仓库更新并合并到本地	git pull origin master

3.3 在 github 或 gitee 上注册并建立一个仓库

1. GitHub 登录成功后的个人主页：



2. 仓库创建成功页面：



3. 远程仓库访问路径：https://github.com/Cspix-dery/git-demo-lab

3.4 完成本地到远程仓库的操作

1. git init 初始化本地 Git 仓库，生成隐藏.git 文件夹：



2. 新建测试文件 test.txt，写入初始文本：



第一行：初始版本内容

3.git add 将 test.txt 从工作区加入暂存区；git status 查看仓库状态，文件处于待提交:

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git add test.txt

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   test.txt
```

4.git commit 将暂存区文件提交到本地仓库，生成 commit 版本:

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git commit -m "首次提交：新建test.txt初始版本"
[main (root-commit) b0aebba] 首次提交：新建test.txt初始版本
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

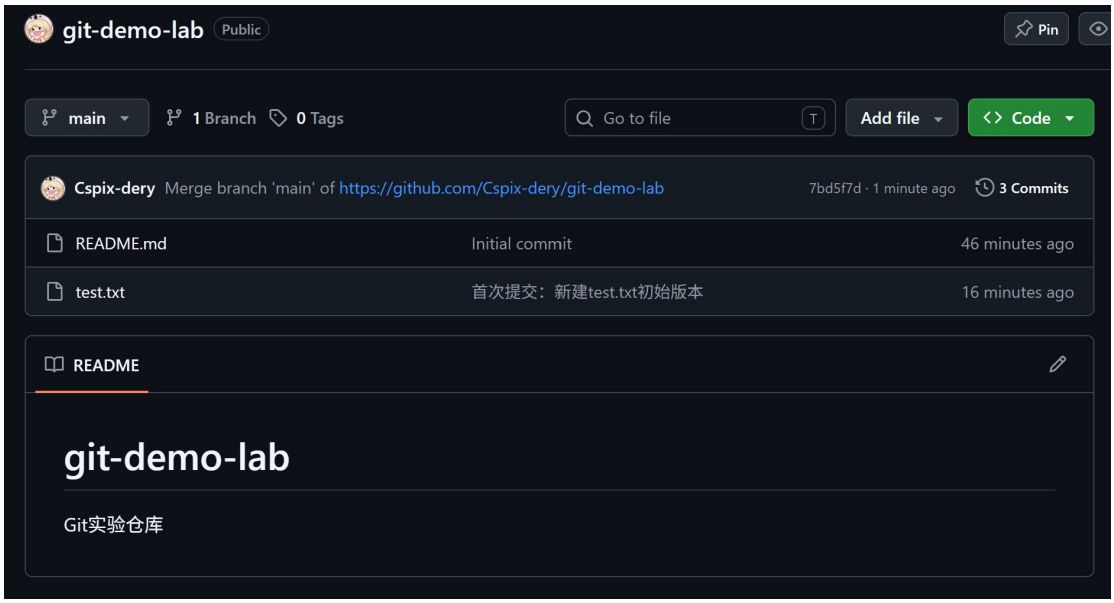
5.关联本地仓库与 GitHub 远程仓库，origin 为远程仓库别名

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git remote add origin https://github.com/Cspix-dery/git-demo-lab.git

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git remote -v
origin  https://github.com/Cspix-dery/git-demo-lab.git (fetch)
origin  https://github.com/Cspix-dery/git-demo-lab.git (push)
```

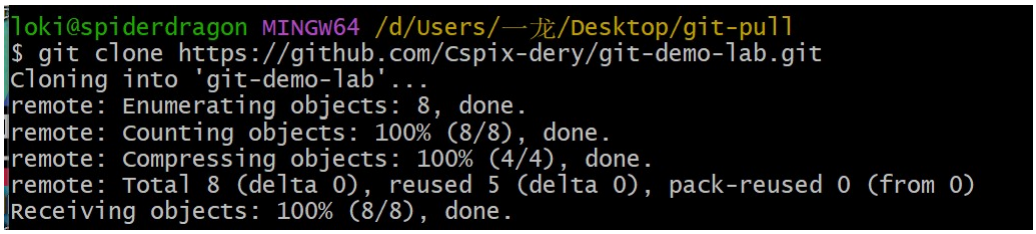
6.git push 将本地 main 分支代码推送到 GitHub 远程仓库。远程仓库已同步 test.txt 文件。

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 22 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 589 bytes | 294.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Cspix-dery/git-demo-lab.git
   095b84f..7bd5f7d  main -> main
branch 'main' set up to track 'origin/main'.
```

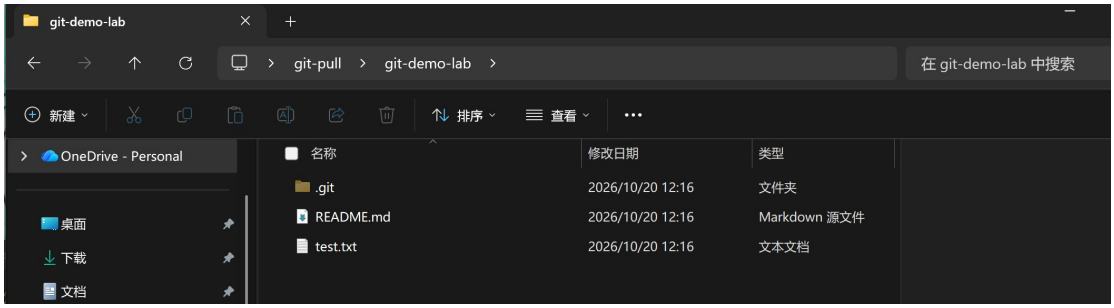


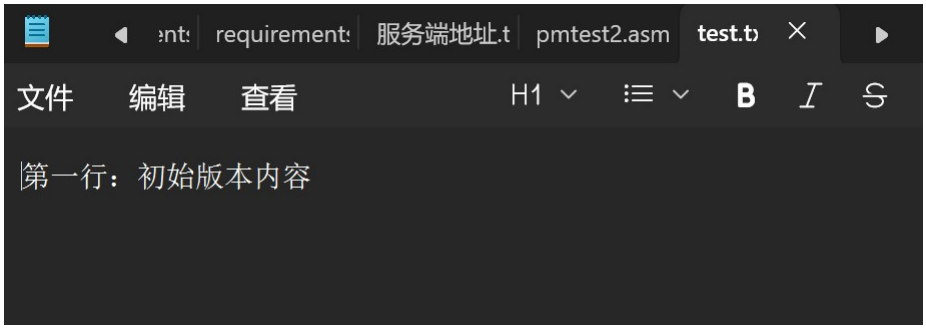
3.5 完成远程仓库到本地的操作

1. git clone 从 GitHub 远程仓库下载完整仓库到本地新目录：



2. 查看克隆得到的 test.txt 内容





3.6 完成分支、合并和冲突管理的操作

第一部分：创建分支，无冲突合并

1.git branch 查看分支:

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git branch
* main
```

2.新建 dev 分支，并切换到 dev 分支:

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git checkout -b dev
Switched to a new branch 'dev'
```

3.在 dev 分支修改 test.txt，增加一行内容:

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (dev)
$ echo "第二行：dev分支新增内容" >> test.txt

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (dev)
$ cat test.txt
第一行：初始版本内容
第二行：dev分支新增内容
```

4.将 dev 分支的修改提交本地仓库:

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (dev)
$ git add test.txt
warning: in the working copy of 'test.txt', LF will be replaced by CRLF the next
 time Git touches it

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (dev)
$ git commit -m "dev分支：增加第二行文本"
[dev 42876a7] dev分支：增加第二行文本
 1 file changed, 1 insertion(+)
```

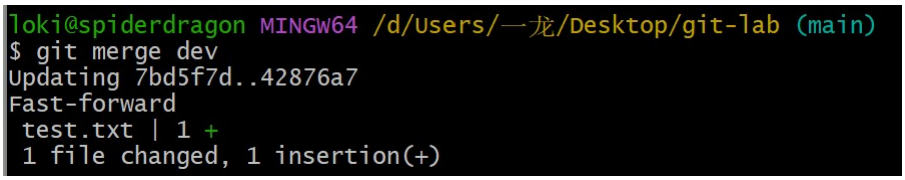
5.切换回 main 主分支，main 分支文件保持原有版本，不受 dev 分支修改影响：

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (dev)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$
```



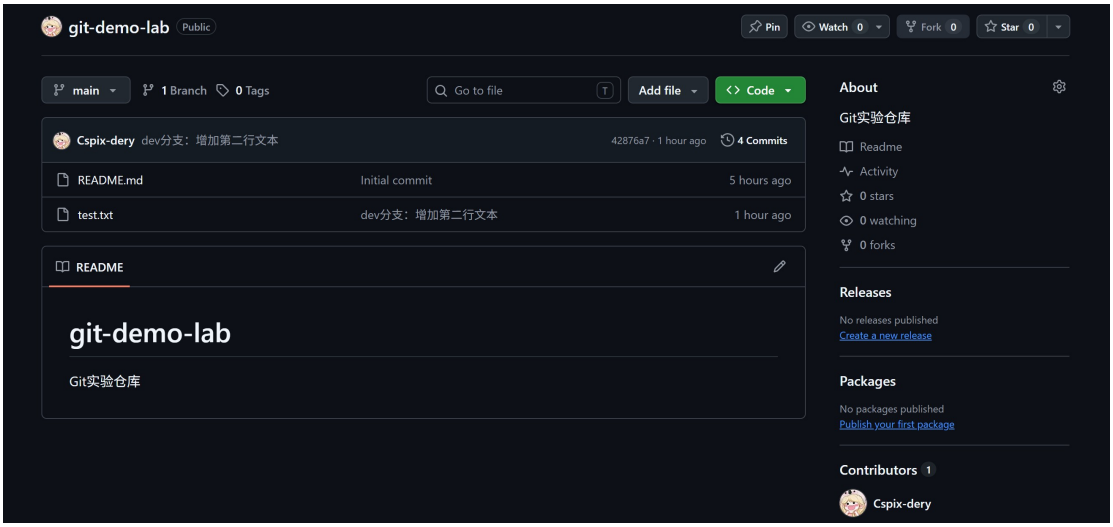
6.main 分支执行 merge 合并 dev 分支,两个分支修改位置不冲突,Git 自动完成合并。查看合并后文件，两行内容都存在：

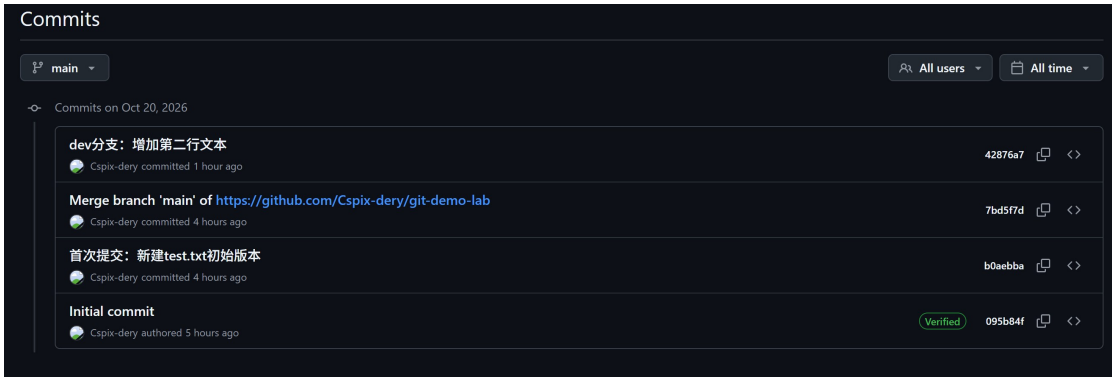




7.push 合并结果到 github：






Commits
main
All users
All time
Commits on Oct 20, 2026
dev分支：增加第二行文本
Cspix-dery committed 1 hour ago
42876a7
Merge branch 'main' of https://github.com/Cspix-dery/git-demo-lab
Cspix-dery committed 4 hours ago
7bd5f7d
首次提交：新建test.txt初始版本
Cspix-dery committed 4 hours ago
b0aebba
Initial commit
Cspix-dery authored 5 hours ago
Verified
095b84f

第二部分：人为制造合并冲突

1.新建 feature 分支，修改 test.txt 第一行内容并提交：

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git checkout -b feature
Switched to a new branch 'feature'

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
$ echo "第一行：feature分支修改这一行！！" > test.txt

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
$ cat test.txt
第一行：feature分支修改这一行！！

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
$ git add test.txt
warning: in the working copy of 'test.txt', LF will be replaced by CRLF the
 time Git touches it

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
$ git add test.txt

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
$ git commit -m "feature分支：修改第一行"
[feature 21a9f93] feature分支：修改第一行
 1 file changed, 1 insertion(+), 2 deletions(-)

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
```

2.切回 main 分支，修改同一个文件同一行，两个分支对同一行做了不同修改，准备产生冲突：

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ echo "第一行：main分支修改这一行！！" > test.txt

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ cat test.txt
第一行：main分支修改这一行！！

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git add test.txt
warning: in the working copy of 'test.txt', LF will be replaced by CRLF the next
 time Git touches it

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git commit -m "main分支：修改第一行文字"
[main a15b0e9] main分支：修改第一行文字
 1 file changed, 1 insertion(+), 2 deletions(-)
```

3.main 分支合并 feature 分支，两个分支修改同一文件同一行，自动合并失败，产生代码冲突，需要人工解决：

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git merge feature
Auto-merging test.txt
CONFLICT (content): Merge conflict in test.txt
Automatic merge failed; fix conflicts and then commit the result.
```

4. 打开 test.txt，Git 使用冲突标记分割两个版本代码。<<<<<< HEAD 代表当前 main 分支内容；=====分割；>>>>>> feature 是待合并分支内容：

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main|MERGING)
$ cat test.txt
<<<<<<< HEAD
第一行：main分支修改这一行！！
=======
第一行：feature分支修改这一行！！
>>>>>>> feature
```

5. 手动编辑文件消除冲突标记，add 提交，完成本次合并：

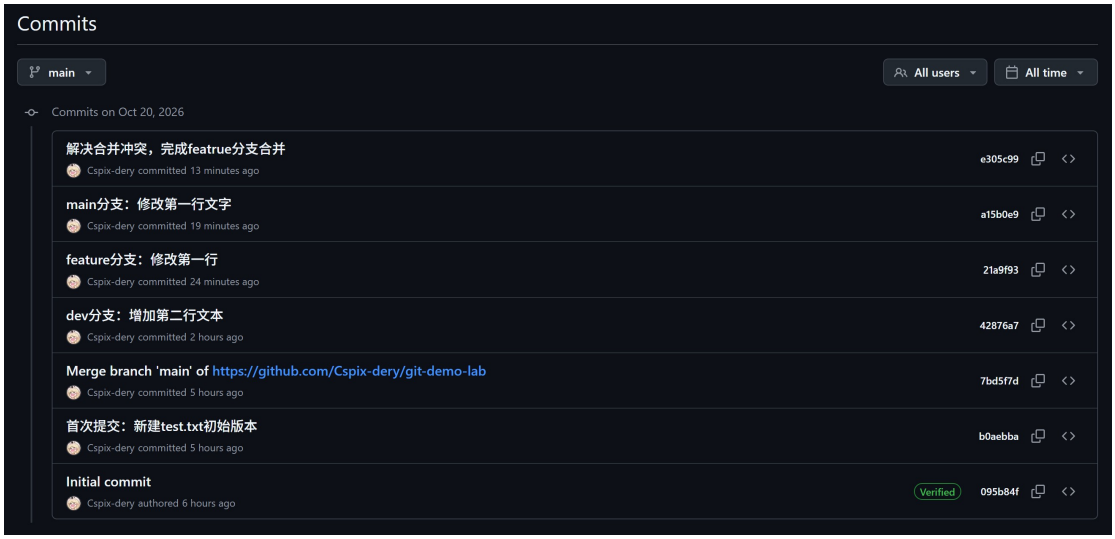



```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main|MERGING)
$ git add test.txt

loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main|MERGING)
$ git commit -m "解决合并冲突，完成featrue分支合并"
[main e305c99] 解决合并冲突，完成featrue分支合并
```

6. push 到远程仓库：





3.7 其他

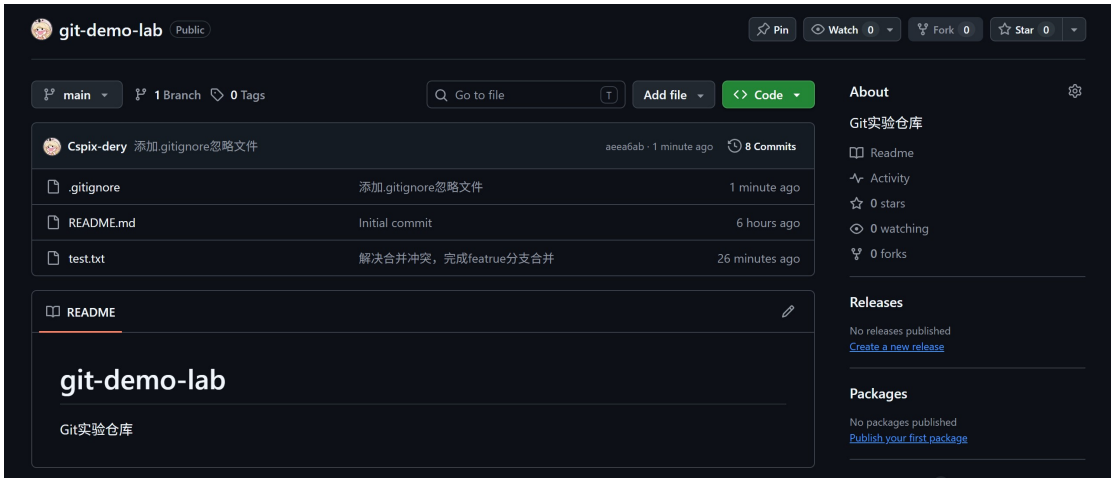
任务一：创建.gitignore 忽略文件

1.创建.gitignore 文件，配置忽略规则，指定 Git 不追踪 log 后缀文件与 temp 文件夹：

```
PS D:\Users\一龙\Desktop\git-lab> echo "*.log" > .gitignore
PS D:\Users\一龙\Desktop\git-lab> echo "temp/" >> .gitignore
PS D:\Users\一龙\Desktop\git-lab> cat .gitignore
*.log
temp/
```

2.提交.gitignore 并推送到远程

```
PS D:\Users\一龙\Desktop\git-lab> git add .gitignore
PS D:\Users\一龙\Desktop\git-lab> git commit -m "添加.gitignore忽略文件"
[main aeea6ab] 添加.gitignore忽略文件
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .gitignore
PS D:\Users\一龙\Desktop\git-lab> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 22 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 370 bytes | 370.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Cspix-dery/git-demo-lab.git
   e305c99..aeea6ab  main -> main
```



任务二：git restore（撤销工作区修改）

git restore test.txt 撤销工作区未暂存的修改，恢复文件到原始版本：

```
PS D:\Users\一龙\Desktop\git-lab> echo "anyelse test" >> test.txt
PS D:\Users\一龙\Desktop\git-lab> cat test.txt
绗﹐蹲琛岈細鍐茬獊瑙 e 嗖瀹屾垚锛屾湭缁堝墠增鏈？
绗﹐篡琛岈細dev鎵嗘敵鑷板﹐鍐呭﹐

anyelse test


PS D:\Users\一龙\Desktop\git-lab> git restore test.txt
PS D:\Users\一龙\Desktop\git-lab> cat test.txt
绗﹐蹲琛岈細鍐茬獊瑙 e 嗖瀹屾垚锛屾湭缁堝墠增鏈？
绗﹐篡琛岈細dev鎵嗘敵鑷板﹐鍐呭﹐
```

4. 实验总结

4.1 实验中遇到的问题及解决方法

问题 1：执行‘git push origin main’时，出现 SSL/TLS 连接失败、无法连接 github.com(https://github.com)的报错：

报错信息：

```
loki@spiderdragon MINGW64 /d/Users/一龙/Desktop/git-lab (main)
$ git push origin main
fatal: unable to access 'https://github.com/Cspix-dery/git-demo-lab.git/': schannel: f
ailed to receive handshake, SSL/TLS connection failed
```

原因分析：国内网络直接访问 GitHub 的 HTTPS 服务器不稳定，SSL 握手超时，无法建立安全连接。

解决方法：关闭 Git 的 SSL 校验（仅用于实验，正式项目不建议），执行 “git config --global http.sslVerify false”，之后重新执行 push。

问题 2：执行 git commit 的时候，提示 Please tell me who you are，无法提交：

报错信息：

```
*** Please tell me who you are.
```

```
Run
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"
to set your account's default identity.
```

原因：Git 没有配置用户名和邮箱，不知道本次提交记录是谁的修改，禁止提交。

解决方法 在 Git Bash 执行两条配置命令：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

配置完成后，重新执行 git commit 即可正常提交。

4.2 实验总结与体会

本次 Git 版本控制实验,我学习并实操了 Git 仓库初始化、文件暂存提交、分支创建切换、远程推送等一系列基础命令，掌握了使用 Git 管理代码版本的基本流程，理解了版本控制工具在项目开发中的作用。

实验过程中我遇到了远程推送 GitHub 失败的问题，报错为 SSL/TLS 连接失败，经过排查，了解到是国内网络访问 GitHub 不稳定导致。通过更换 Gitee 作为远程仓库，修改远程仓库地址后成功完成代码推送，这让我意识到，在实操中除了记住命令，还要学会处理网络、环境带来的突发问题，不能只停留在课本上的理想流程。

在练习分支操作的过程中，我体会到分支机制的优势：可以在不影响主分支稳定代码的前提下，独立开发新功能、进行修改测试，适合多人协作开发。同时我也明白了 Git 工作区、暂存区、本地仓库、远程仓库四者之间的数据流转关系，理解了 add、commit、push 等命令各自负责的环节，纠正了之前对版本提交流程模糊的认知。

这次实验也让我认识到规范使用版本管理的重要性。清晰的提交备注、合理的分支规划，能够方便追溯每一次修改，便于回溯版本和协同开发。Git 命令较多，单纯死记硬背很容易混淆，只有多动手实操、主动排查报错，才能真正理解每一条命令的含义。后续我会继续熟练 Git 的常用操作，把版本控制运用到之后的编程项目里，养成良好的代码管理习惯。